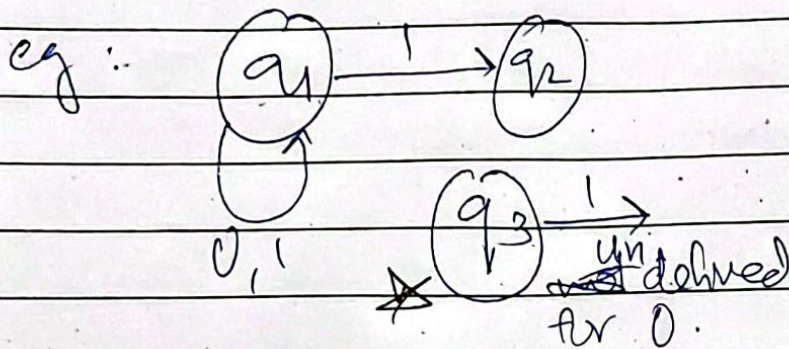


Nondeterministic finite Automata NFA

Machine can be in multiple states at any given time.



what should it do when it is in q_1 and reads 1?

is it probabilistic or is it based on context?

→ this is not a probabilistic machine. No probabilities involved here.

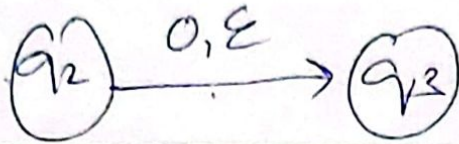
* → either this case won't occur (reading 0 at q_3)
→ we don't care what happens when this occurs.

Number of outgoing arrows = number of characters in alphabet.
→ important for DFA.

DFA:
deterministic finite automata

machine is in only 1 possible state at any given time

well known by looking at transition table or diagram

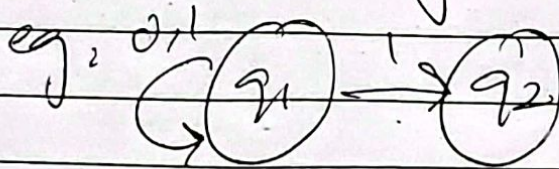


Date: _____

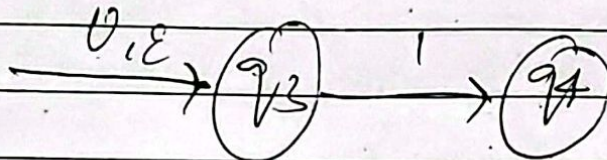
ϵ on arrow is 'empty transition' without reading anything it can go from q_2 to q_3 . ~~without reading anything as soon as it starts it can go~~

3 cases of NFA:

→ multiple transitions for same character on single state

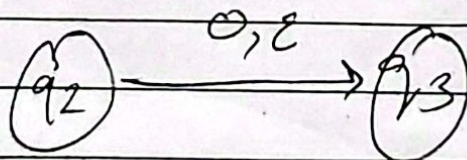


→ no of outgoing arrow $\neq$ no. of characters in alphabet.

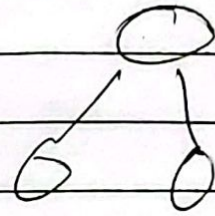


q_3 ~~not defined~~ undefined for 0.

→ empty transition.



At a nondeterministic instance the machine makes 2 copies itself - one stays in q_1 and the other is in q_2 . The next input is treated accordingly by both machines depending on which state they are at.



The depth of this tree is at max the length of string. Some branches can die when it reaches a point where it isn't defined for an input.

If any one branch is accepting then program is accepted. else rejected. If none of the branches end in an accepting state.

We will always go to the end of entire string before deciding if a state is being accepted or not.

diff b/w definition of NFA is only ϵ . ϵ becomes part of input somehow even if it is not part of alphabet.

eg $\Sigma = \{a, b\}$
 some string over Σ . $a, ab, aba, \dots$
 concatenation of a, b and a .

ae ea aea

empty string at end, middle, beginning
does nothing at all. Neither
length nor the string itself.

At end of δ we don't have a set,
we have a powerset $P(\emptyset)$.

What is a powerset of a set? $P(\emptyset)$?
any set (finite/infinite), then collection
of all possible subsets is called
powerset.

$$a = \{1, 2, 3\}$$

$$P(a) = \{ \emptyset, \{1\}, \{2\}, \{3\}, \{1, 2\}, \{1, 3\}, \{2, 3\}, \{1, 2, 3\} \}$$

↓
null set $\emptyset$

no of elements in set = n .

no of elements in powerset = 2^n

$$a = 3$$

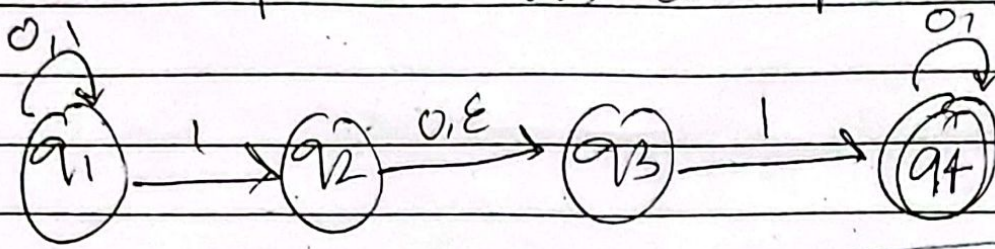
$$P(a) = 8 = 2^3$$

in NFA the transition table will have
 $\{ \}$ sets and not empty states.

anything not defined is a null set.

writing q_1 is wrong
 $\{q_1\}$ is right.

11 accepted as sample ϵ in Lho. 1 & 1.



$\Sigma = \{0, 1\}$

$L = \{x \mid 1 \text{ appears in the last position of } x\}$

In L :

111
100
1100

Not in L :

ϵ (empty string)
1
11 000
0
00

All NFAs have an equivalent DFA.

If a string is rejected by either or accepted by either, the same for the other.

Same computational power of both NFA and DFA.

Prove that class of regular languages is closed under union and concatenation ~~for both~~ using NFA:

for union operation:

Suppose A and B are two regular lang^s. Then there must exist DFA for both A and B.

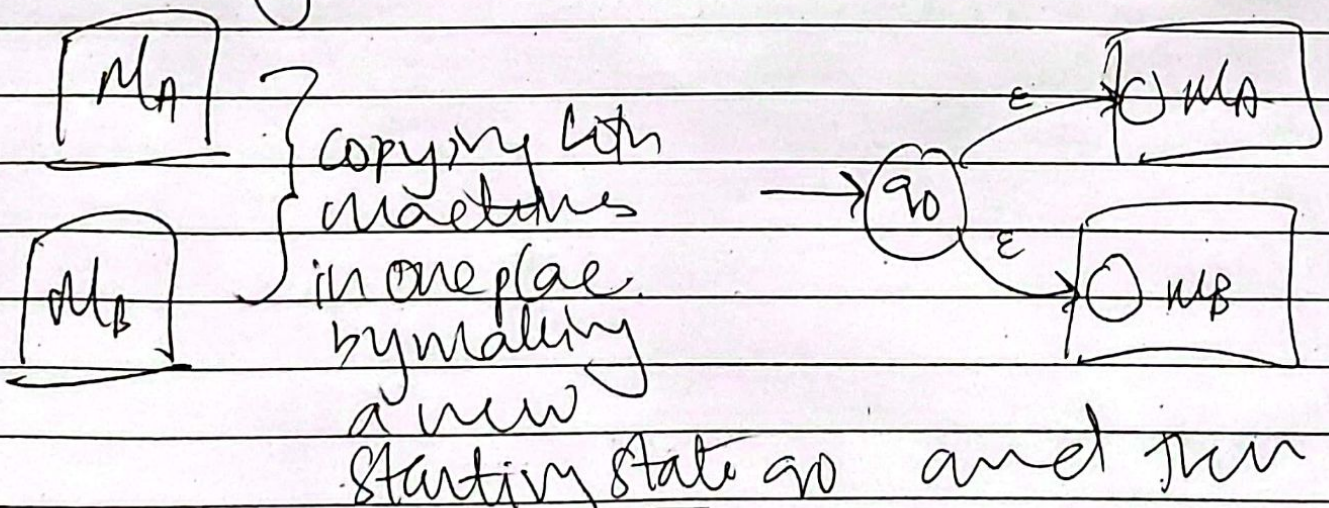
M_A is a DFA for A.

M_B is a DFA for B.

Suppose $C = A \cup B$, then C is also regular. and if regular we can create DFA for C. if DFA exists for C then C is a regular lang^y.

All we gotta do is construct a DFA for C..

Creating M_C for C:



Starts state of
Connect starting state of q_0 to M_A and to M_B using empty string.

connecting q_0 to starting state of M_A and M_B at the same time.

combined 2 DFAs and a new initial state to form NFA.

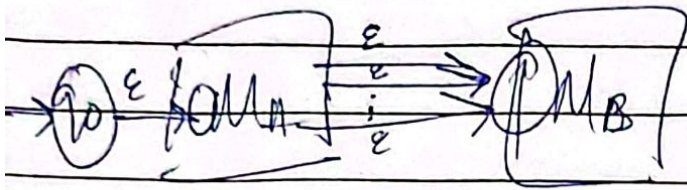
if string will either be accepted by 1 or both. when string ends check if its in accepting state or rejecting state.

concatenation operation:

proof: let A and B be two regular languages, then there exists M_A and M_B (which are DFA's)

Now consider the lang $C = AB$.

if we could ~~show~~ show that we can construct a machine that accepts C . then we are done.



connect an initial state go to initial state of M_1 . Remove any final states from M_1 . and connect (however many) final states of M_1 to initial states of M_2 using empty strings.

concatenation. All strings in C are all strings concatenating A & B .

$N \rightarrow$ NFA machine
 $M \rightarrow$ DFA machine
Date: _____

$NFA \equiv DFA$

Proof:

Let's start from some NFA N . we need a DFA M which needs to have the same language as N .

'powers suggest these are new sets'

M contains powersets of sets of N .

(no. of states grows exponentially.

n in NFA. } 2^n is
 $O(2^n)$ in DFA. } upper bound.
for Q .

not all states useful or reachable.
When we reach conversion we delete these unwanted states.

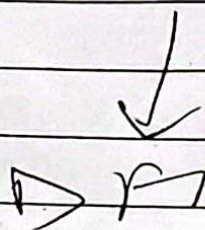
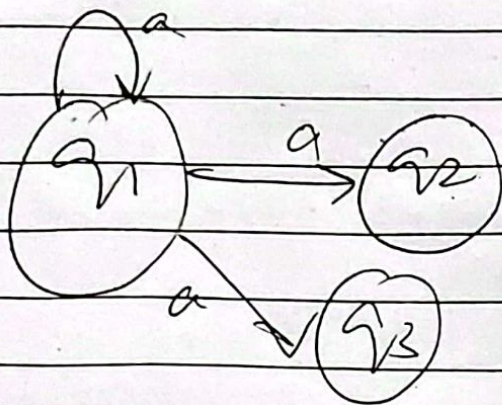
$\cup$ means union.

$$\delta'(r, a) = \bigcup_{r \in R} \delta(r, a)$$

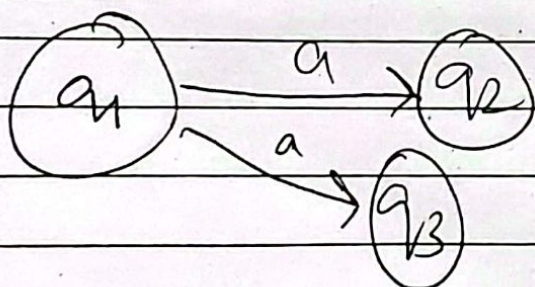
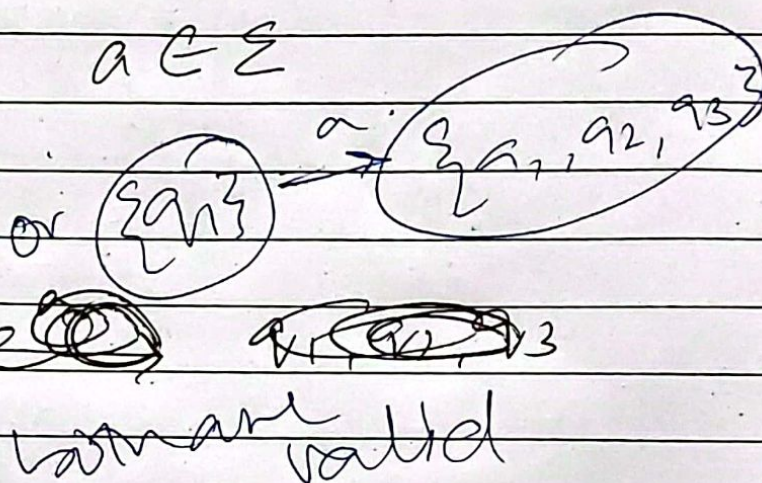
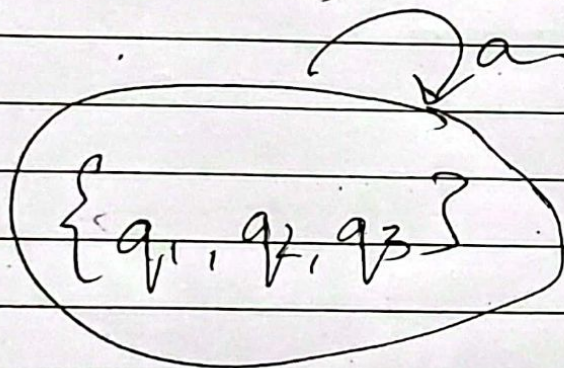
union of multiple things

This union must exist in $\mathcal{L}$!

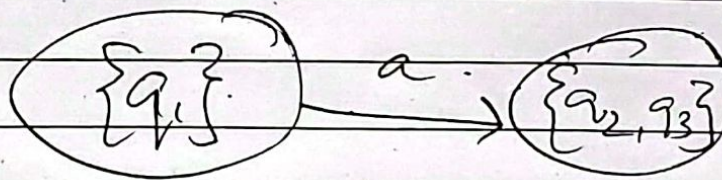
eg. a partial NFA is given



DFA - $a \in \Sigma$



NFA -



DFA

$q2, q3$
must
exist.